

MANUAL TÉCNICO

Aplicación de Rony Alvarez - Proyecto 1- EDD-

Índice

Estructuras de datos.....	4
1. Lista doblemente enlazada.....	4
2. Lista circular doblemente enlazada.....	5
3. Lista circular enlazada.....	7
4. Lista enlazada simple.....	8
Cargar el archivo .java.....	8
Análisis del archivo de entrada.....	9
Autómata finito(la vida del analizador léxico).....	11
Guardar Código Java.....	13
Generar reportes.....	14
Generación de la sintaxis y producciones.....	16
Diagramas de flujo.....	22
i. Análisis del archivo .java.....	22
ii. Generación de reportes.....	23
iii. Generación de traducción.....	24

Introducción

Este manual acerca del software del Med trak, está hecho para que los desarrolladores interesados en el programa sean capaces de entender la lógica detrás de su funcionamiento, los métodos y funciones usadas en lenguaje de programación Perl.

Objetivo

El objetivo de este manual es ayudar a los desarrolladores, tutores auxiliares, y público en general a comprender el desarrollo de este software, y el pensamiento del desarrollador que lo ha creado.

También está hecho con el objetivo de mostrar al público general que el estudiante de EDD ha logrado utilizar los conceptos que fueron planteados antes del práctica, tanto como respetar las reglas de las estructuras que se nos fueron permitidas para usarse.

Dirigidos

Este manual está orientado a todas aquellas personas que estén interesadas en el funcionamiento del software, el desarrollo y lógica detrás de él. Para que en un futuro, por si les es de utilidad puedan utilizar este programa como guía para sus actividades.

Especificación técnica

Requisitos de Hardware

- Computadora de escritorio o portátil.
- 4GB de memoria RAM.
- 256GB de disco duro o más.
- Procesador intel i3, AMD ryzen 3, o superior.
- Resolución gráfica mínima de 1024 x 768 píxeles.

Requisitos de Software

- S.O. Windows 10 o superior.
- Tener instalado Perl Strawberry.
- Tener instalado GraphViz.
- Lenguaje de Programación Perl.

Especificación técnica

Estructuras de datos

1. Lista doblemente enlazada

Esta estructura de datos fue creada 100% desde 0 en el lenguaje de programación Perl, se utilizó la Poo para crear la clase desde 0 y así tener total control de la misma. La lista doblemente enlazada consiste en un conjunto de elementos nodos que están conectados entre sí por medio de enlaces entre el que les sigue y el que los precede.

Esta es la clase nodo.pl

```
package nodo;
use strict;
use warnings;

sub new{
    my ($class, $valor)=@_;
    my $self = {
        valor => $valor,
        siguiente => undef,
        anterior => undef,
    };
    return bless $self, $class;
}

# getters / setters
sub value { $_[0]->{valor} }
sub set_value { $_[0]->{valor} = $_[1] }
sub prev { $_[0]->{anterior} }
sub set_prev { $_[0]->{anterior} = $_[1] }
sub next { $_[0]->{siguiente} }
sub set_next { $_[0]->{siguiente} = $_[1] }
1;
```

Esta otra, ya es la clase listaDoblementeEnlazada.

```

package listaDoblementeEnlazada;
use strict;
use warnings;

#llamar al nodo
require 'nodo.pl';

sub new{
    my($class) = @_ ;
    my($self) = {
        head => undef,
        tail => undef,
        size => 0,
    };
    return bless $self, $class;
}

#Método para ver si está vacía
sub estaVacía{ $_[0]->{size} == 0}

#método para ver el tamaño total
sub tamaño{ $_[0]->{size}}

```

Esta estructura de datos, además, cuenta con clases para insertar y eliminar elementos por delante o por detrás de la lista, un método para iterar la lista y otro para encontrar un elemento dentro de la lista.

2. Lista circular doblemente enlazada

Esta estructura de datos fue creada 100% desde 0 en el lenguaje de programación Perl, se utilizó la Poo para crear la clase desde 0 y así tener total control de la misma. La lista circular doblemente enlazada consiste en un conjunto de elementos nodos que están conectados entre sí por medio de enlaces entre el que les sigue y el que los precede, pero, como detalle extra, el nodo del final y el nodo del principio, están conectados entre sí.

Esta es la clase de [nodoCircularDoble.pl](#)

```

package nodoCircularDoble;
use strict;
use warnings;

sub new {
    my ($class, $valor) = @_;
    my $self = {
        valor      => $valor,
        siguiente => undef,
        anterior  => undef,
    };
    return bless $self, $class;
}

```

Y esta es la clase de [listaCircularDoble.pl](#)

```

package listaCircularDoble;
use strict;
use warnings;

require 'nodoCircularDoble.pl';

sub new{
    my($class) = @_;
    my $self = {
        head => undef,
        size=>0,
    };
    return bless $self, $class;
}

```

Esta estructura de datos, además, cuenta con clases para insertar por detrás, dos métodos para iterar la lista por delante y por detrás y otro para encontrar la cola de la lista, para la mejor manipulación de datos en el futuro.

3. Lista circular enlazada

Esta estructura de datos fue creada 100% desde 0 en el lenguaje de programación Perl, se utilizó la Poo para crear la clase desde 0 y así tener total control de la misma. La lista circular enlazada consiste en un conjunto de elementos nodos que están conectados entre sí por medio de enlaces con el nodo que les sigue pero, como detalle extra, el nodo del final y el nodo del principio, están conectados entre sí.

Esta es la clase de [nodoCircular.pl](#)

```
package nodoCircular;
use strict;
use warnings;

sub new{
    my($class, $valor) = @_;
    my($self) = {
        valor => $valor,
        siguiente => undef,
    };
    return bless $self, $class;
}
```

Esta es la clase de [listaCircular.pl](#)

```
package listaCircular;
use strict;
use warnings;
require 'nodoCircular.pl';

sub new{
    my($class) = @_;
    my $self = {
        head => undef,
        size => 0,
    };
    return bless $self, $class;
}
```

Esta estructura de datos, además, cuenta con clases para insertar por detrás, un método para iterar la lista por delante, uno para encontrar el tamaño de la lista y otro para encontrar la cola de la lista, para la mejor manipulación de datos en el futuro.

4. Lista enlazada simple

Esta estructura de datos fue creada 100% desde 0 en el lenguaje de programación Perl, se utilizó la Poo para crear la clase desde 0 y así tener total control de la misma. La lista enlazada simple consiste en un conjunto de elementos nodos que están conectados entre sí por medio de enlaces con el nodo que les sigue, es la estructura más simple de este proyecto.

Esta es la clase de [nodoSimple.pl](#)

```
package nodoSimple;
use strict;
use warnings;

sub new{
    my($class, $valor) = @_;
    my $self = {
        valor => $valor,
        siguiente => undef,
    };
    return bless $self, $class;
}
```

Esta es la clase de [listaSimple.pm](#)

```
package listaSimple;
use strict;
use warnings;

require 'nodoSimple.pl';

sub new{
    my($class) = @_;
    my $self = {
        head => undef,
        size => 0,
    };
    return bless $self, $class;
}
```

Esta estructura de datos, además, cuenta con clases para insertar por detrás y por delante, un método para iterar la lista por delante y uno para encontrar el tamaño de la lista para la mejor manipulación de datos en el futuro.

Sub para registrar un nuevo medicamento

Para registrar un nuevo medicamento y que tenga un código de medicamento correctamente ordenado, el sistema se encarga de generar el código del medicamento automáticamente, el usuario solo debe ingresar los demás datos del medicamento y luego de eso, se guardará el mismo.

```
#----- 1. SUB PARA REGISTRAR MEDICAMENTOS-----  
sub RegistroMedicamento{  
    #Ver la estructura del código MED00X  
    #Acá vamos a registrar medicinas para usar  
    print "\n*****Registro de medicamentos*****\n";  
    my $codigo = generar_codigo_medamento($listaMedicamentos);  
    print "Nombre comercial del medicamento: "; chomp(my $nombre = <STDIN>);  
    print "Principio activo del medicamento: "; chomp(my $activo = <STDIN>);  
    print "Laboratorio fabricante del medicamento: "; chomp(my $laboratorio = <STDIN>);  
    print "Cantidad de stock del medicamento: "; chomp(my $stock = <STDIN>);  
    print "Fecha de vencimiento del medicamento: "; chomp(my $fecha = <STDIN>);  
    print "precio del medicamento: "; chomp(my $precio = <STDIN>);  
    print "Nivel mínimo de reorden del medicamento: "; chomp(my $nivel = <STDIN>);  
    print "\n*****Muchas gracias por tus respuestas :D!*****\n";  
  
    my $medicina = medicamento->new(  
        codigoMedicina => $codigo,  
        nombreComercial => $nombre,  
        principioActivo => $activo,  
        laboratorioFabricante => $laboratorio,  
        cantidadStock => $stock,  
        fechaVencimiento => $fecha,  
        precio => $precio,  
        nivelMinimoReorden => $nivel,  
    );  
  
    #Meter el objeto a la lista doblemente enlazada, al fondo  
    $listaMedicamentos->pushBack($medicina);  
    print "\n*****Medicamento registrado :D!*****\n";  
}  
  
#-----SUB PARA CREAR CODIGO MED-----  
sub generar_codigo_medamento {  
    my ($listaMedicamentos) = @_;  
    my $num = $listaMedicamentos->size + 1; #para contar desde uno cawn  
    my $codigo = sprintf("MED%03d", $num);  
    return $codigo; }
```

Registrar medicamentos por carga .csv

En este sub, el usuario debe de colocar una ruta válida para cargar al sistema el archivo .csv, luego de eso, mientras se van cargando los datos, el sistema comprueba si el código de medicamento ya existe dentro del sistema, y si es así, el sistema automáticamente correrá los códigos de los medicamentos, para que todo quede ordenado.

```

#-----2. SUB PARA CARGA MASIVA DE MEDICAMENTOS-----
sub cargaMasivaMedicina{
    my($ruta, $listaMedicamentos)=@_;

    my $csv = Text::CSV->new({ binary => 1, auto_diag => 1 }); open my $fh, "<:encoding(utf8)", $ruta or die "Lo lamento, no se
    # Si el archivo tiene encabezados, se usan de header
    my $header = $csv->getline($fh);

    #Recorrer todas las líneas
    while(my $row = $csv->getline($fh)){
        #el row es un arrayRef con toda la data de los medicamentos
        my ($codigo, $nombre, $activo, $laboratorio, $stock, $fecha, $precio, $nivel) = @$row;

        my $existe = 0;
        $listaMedicamentos->iterar(sub {
            my $nodo = shift;
            my $med = $nodo->value; # método del setter para obtener el contenido
            #verificar si el código está en la lista
            if($med->codigoMedicina eq $codigo){
                $existe = 1;
            }
        });
        #si es que hay un código repetido, creamos otro
        if ($existe) {
            $codigo =~ /MED(\d+)/;
            my $num = $1;
            $num++;
            $codigo = sprintf("MED%03d", $num); }
        # Crear objeto medicamento
        my $medicina = medicamento->new(
            codigoMedicina => $codigo,
            nombreComercial => $nombre

```

Autómata finito(la vida del analizador léxico)

En este sub, el administrador se encarga de subir los datos de cada proveedor al sistema, luego de rellenar los campos, se guardan los datos dentro de la lista circular simple.

```

#-----3. SUB PARA CREAR PROVEEDORES EN EL SISTEMA-----
sub RegistroProveedor{
    #Acá vamos a registrar medicinas para usar
    print "\n*****Registro de proveedores*****\n";
    print "NIT del proveedor: "; chomp(my $nit = <STDIN>);
    print "Nombre de la empresa: "; chomp(my $nombre = <STDIN>);
    print "Contacto principal: "; chomp(my $contacto = <STDIN>);
    print "Teléfono: "; chomp(my $telefono = <STDIN>);
    print "Dirección: "; chomp(my $direccion = <STDIN>);
    print "\n*****Muchas gracias por tus respuestas :D*****\n";

    my $proveedor = proveedor->new(
        nit => $nit,
        nombreEmpresa => $nombre,
        contactoPrincipal => $contacto,
        telefono => $telefono,
        direccion => $direccion,
    );

    #Meter el objeto a la lista doblemente enlazada, al fondo
    $listaProveedores->insertar($proveedor);
    print "\n*****Proveedor registrado :D*****\n";

    print "\n--- Lista de proveedores ---\n";
    $listaProveedores->recorrer(sub {
        my $nodo = shift;
        my $prov = $nodo->valor; # método del setter para obtener el contenido
        print $prov->nit, " - ", $prov->nombreEmpresa, "\n";
    });
}

```

Insertar nueva entrega al proveedor

Para este sub, primero se verifica que el nit del proveedor se encuentre dentro de nuestra base de datos, si el nit es encontrado, se le piden los datos al administrador para crear el registro correspondiente. Si el código de la medicina ingresada no está dentro de nuestra lista Doblemente enlazada, la operación se cancelará, pero, si se encuentra dentro de la lista, se agregaron esas unidades extra a nuestro stock.

```
#----- 4. SUB PARA INSERTAR NUEVA ENTREGA A PROVEEDOR-----
sub registrarEntregaProveedor{
    my $encontrado = 0;
    print "\n*****Registro de nueva entrega a proveedor*****\n";
    print "Por favor, ingresa el nit del proveedor: "; chomp(my $nit = <STDIN>);
    print "+++Buscando+++++\n";
    print "+++Buscando+++++\n";
    print "+++Buscando+++++\n";
    print "+++Buscando+++++\n";
    #recorremos y buscamos el número de nit del proveedor
    $listaProveedores->recorrer(sub {
        my $nodo = shift;
        my $prov = $nodo->valor; # método del setter para obtener el contenido

        if($prov->nit eq $nit){
            #ver si encontraron un proveedor
            $encontrado = 1;
            #si encuentra el nit, pediremos los demás datos
            print "\n*****Registro de nueva entrega a proveedor*****\n";
            print "Por favor, ingresa el número de la factura: "; chomp(my $factura = <STDIN>);
            print "Por favor, ingresa la fecha de entrega del medicamento: "; chomp(my $fecha = <STDIN>);
            print "Por favor, ingresa el código del medicamento: "; chomp(my $codigo = <STDIN>);
            print "Por favor, ingresa la cantidad entregada del medicamento: "; chomp(my $cantidad = <STDIN>);
            my $existe = 0;
            $listaMedicamentos->iterar(sub {
                my $nodo = shift;
                my $med = $nodo->value; # método del setter para obtener el contenido
```

Generar solicitud de reabastecimiento

Para este sub, verificamos el departamento al que pertenece el usuario, luego, procederemos a pedirle los datos necesarios para poder ingresar la solicitud al sistema, osea, dentro de la lista circular doblemente enlazada.

```
#----- 5. SUB PARA SOLICITUD DE REABASTECIMIENTO-----
sub realizarSolicitudReabaste{
    print "\n*****Nueva solicitud de reabastecimiento*****\n";
    print "Por favor, ingresa el código del medicamento requerido: "; chomp(my $medicamento = <STDIN>);
    print "Por favor, ingresa la cantidad del medicamento requerido: "; chomp(my $cantidad = <STDIN>);
    print "\n*****Muchas gracias por tus respuestas :D!*****\n";
    my $t = localtime;
    my $fecha = $t->ymd;
    my $encontrado = 0;

    $listaMedicamentos->iterar(sub {
        my $nodo = shift;
        my $med = $nodo->value; # método del setter para obtener el contenido
        #verificar que la medicina exista para poder crear la solicitud
        if($med->codigoMedicina eq $medicamento){
            $encontrado = 1;
        }
    });

    if($encontrado == 1){
        my $codigo = generar_codigo_solicitud($listaSolicitudesReabaste);
        #si lo encuentra, se guarda la solicitud
        my $solicitud = solicitudReabastecimiento->new(
            codigoSolicitud => $codigo,
            departamento => $rolGeneral,
            medicamentoRequerido => $medicamento,
            cantidadSolicitada => $cantidad,
```

Aceptar o rechazar solicitudes.

En este sub se muestran todas las solicitudes que hay en el sistema, las solicitudes van apareciendo de una en una, y siempre se pregunta al administrador si desea aceptar o rechazar la solicitud. En caso de aceptarla, se verifica antes si hay suficientes unidades disponibles de este medicamento, si no hay suficientes, se rechaza la solicitud. Las solicitudes se recorren en su orden de llegada.

```
#----- 5. SUB PARA CONFIRMAR SOLICITUD DE REABASTECIMIENTO-----
sub aprobarSolicitudReabaste{
    $listaSolicitudesReabaste->recorrerAdelante(
        sub {
            #Acá vamos a mostrar la información de la solicitud
            my $nodo = shift;
            my $solicitud = $nodo->valor;
            #mostrar las solicitudes que no han sido confirmadas
            if($solicitud->{estadoSolicitud} eq 'sin confirmar'){
                print "\n--- Solicitudes disponibles: ", $listaSolicitudesReabaste->{size}, "---\n";
                print $solicitud->departamento, " - ", $solicitud->fechaSolicitud, " - ", "\n", $solicitud->medicamentoRequerido,
                print "Aprobaremos esta solicitud? :D (s/n): "; chomp(my $respuesta = <STDIN>);
                #si se aprueba la solicitud, se busca el código del medicamento en la lista de medicinas
                #se establece el estado en aceptado
                if(lc($respuesta) eq 's'){
                    #recorrer los medicamentos disponibles
                    print "\n--- Buscando unidades disponibles ---\n";
                    $listaMedicamentos->iterar(sub {
                        my $nodo = shift;
                        my $med = $nodo->value; # método del setter para obtener el contenido
                        #si los códigos de los medicamentos son iguales, verificar si hay unidades suficientes
                        if($med->{codigoMedicina} eq $solicitud->{medicamentoRequerido}){
                            #verificar que en la solicitud hayan pedido menos unidades que las disponibles
                        }
                    });
                }
            }
        }
    );
}
```

Visualizar inventario completo

En este sub se muestran todas las medicinas que hay dentro de los registros de la lista doblemente enlazada. Se van mostrando todas las medicinas, con todos sus datos respectivamente. A su vez, el sistema va marcando automáticamente si una medicina tiene stock suficiente, si falta stock o si está próximo a vencer determinado medicamento, comparando la fecha de hoy con la fecha de vencimiento.

```
sub visualizarInventario{
    print "\n--- Lista de medicamentos en nuestro inventario---\n";
    print "\n--- Esto es todo lo que tenemos dentro del hospital (..)---\n";

    my $fecha = localtime;
    $listaMedicamentos->iterar(sub {
        my $nodo = shift;
        my $med = $nodo->value; # método del setter para obtener el contenido
        print $med->codigoMedicina, " - ", $med->nombreComercial, " - ", $med->principioActivo, " - ", $med->laboratorio, "\n";
        #verificar si se necesita comprar más
        if($med->cantidadStock < $med->nivelMinimoReorden){
            print "----->>>> ALERTA : Se necesita solicitar más unidades de ", $med->nombreComercial, "\n";
        }
        #verificar si está próximo a vencer
        my $fecha_vencimiento_str = $med->fechaVencimiento;
        my $fecha_vencimiento = Time::Piece->strptime($fecha_vencimiento_str, "%Y-%m-%d");
        if ($fecha_vencimiento < $fecha){
            print "----->>>>Lo siento, pero, el medicamento ya venció.\n";
        } elsif ($fecha_vencimiento <= $fecha + ONE_DAY*7){
            print "----->>>>>>>>>>Ten mucho cuidado!, el medicamento vence en menos de una semana.\n";
        } else {
            print "----->>>(0u0!), Yay!, el medicamento aun esta vigente. :) \n";
        }
        print "\n-----\n";
    });
}
```

Visualizar inventario completo - departamental

En este sub se muestran todas las medicinas que hay dentro de los registros de la lista doblemente enlazada. Se le pide al usuario que indique si desea buscar el medicamento por identificador o por nombre, luego se consulta si el medicamento existe, si ese fuese el caso, se muestran los datos del medicamento, de lo contrario, se muestra un mensaje de error.

```
#-----SUB PARA VISUALIZAR INVENTARIO - DEPARTAMENTAL-----
sub visualizarInventarioDepartamental{
    my $encontrado = 0;
    print "\n--- Lista de medicamentos en nuestro inventario---\n";
    print "\n*****Acá podrás buscar el medicamento de tu interes!*****\n";
    print "Cuentame!, Quieres buscar el medicamento por nombre o identificador? :D, n para nombre e i para identificador (n/i): ";
    if(lc($respuesta) eq 'i'){
        print "\n--- Lista de medicamentos ---\n";
        print "Ingresa el código del medicamento del que necesitas información!: "; chomp(my $codigo = <STDIN>);
        $listaMedicamentos->iterar(sub {
            my $nodo = shift;
            my $med = $nodo->value; # método del setter para obtener el contenido
            if($med->codigoMedicina eq uc($codigo)){
                print $med->nombreComercial, " - ", $med->cantidadStock, "\n";
                $encontrado = 1;
            }
        });
        print "\n*****Oye!, ese medicamento no existe*****\n" unless $encontrado;
    }
    elsif(lc($respuesta) eq 'n'){
        print "\n--- Lista de medicamentos ---\n";
        print "Ingresa el nombre del medicamento del que necesitas información!: "; chomp(my $nombre = <STDIN>);
        $listaMedicamentos->iterar(sub {
            my $nodo = shift;
```

Reportes gráficos - GraphViz

En esta parte, se muestra un pequeño menú, donde se puede elegir qué tipo de reporte generar, existen 4 tipos diferentes de reportes y todos llevan una estructura bastante similar, lo que varía en cada uno de ellos, es la cantidad de aristas que surgen de cada uno, y la matriz que es un caso especial.

Imagen del menú.

```
sub menu_admin{
    while(1){
        elsif($op eq '8'){
            print "\n*****Creación de reportes gráficos*****\n";
            print "Que reporte deseas generar? \n";
            print "1.) Reporte de medicinas\n";
            print "2.) Reporte de solicitudes\n";
            print "3.) Reporte de proveedores\n";
            print "4.) Reporte comparación\n";
            print "5.) salir :(\n";
            print "Que reporte deseas generar?: "; chomp(my $opcion = <STDIN>);
            last if $opcion eq '5';
            if($opcion eq '1'){
                graficarListaMedicina($listaMedicamentos);
            }
            elsif($opcion eq '2'){
                graficarListaSolicitudes($listaSolicitudesReabaste);
            }
            elsif($opcion eq '3'){
                graficarListaProveedores($listaProveedores);
            }
            elsif($opcion eq '4'){
            }
            else{
                print "!!!!!!!!!!!!!!OYE, esa opción no existe!!!!!!!!!!!!!!";
            }
        }
        else{
            print "lo siento mucho!, pero esa opción no existe en este menú.";
        }
    }
}
```

Imagen de ejemplo de una función generativa de reportes.

```
665 sub graficarListasolicitudes{  
666     my($listaSolicitudesReabaste)=@_  
667     my $g = GraphViz->new(directed =>1);  
668     my $fecha = localtime;  
669     my $ordenSolicitud = 0;  
670  
671     #hay que recorrer la lista de medicinas  
672     $listaSolicitudesReabaste->recorrerAdelante(sub {  
673         my $nodo = shift;  
674         my $solicitud = $nodo->valor; # método del setter para obtener el contenido  
675         $ordenSolicitud++;  
676         my $color = ($ordenSolicitud == 1) ? 'red' : 'white';  
677  
678         if($solicitud->estadoSolicitud eq 'sin confirmar'){  
679             ■ #cada nodo será un rectángulo con color  
680             $g->add_node(  
681                 $solicitud->{codigoSolicitud},  
682                 label => "$ordenSolicitud\n$solicitud->{departamento}\n$solicitud->{medicamentoRequerido}\n$solicitud->{  
683                     shape => 'circle',  
684                     style => 'filled',  
685                     fillcolor => $color,  
686                     color => 'black',  
687                     fontcolor => 'black'  
688                 );  
689             # Conexión hacia el siguiente  
690             if ($nodo->siguiente) {  
691                 $g->add_edge($solicitud->{codigoSolicitud} => $nodo->siguiente->valor->{codigoSolicitud});  
692             } # Conexión hacia el anterior (en rojo)  
693         }
```

Solamente se van creando los nodos, mientras vamos recorriendo las estructuras de datos, luego de eso, (dependiendo de la estructura que estemos recorriendo), vamos agregando aristas para conectar los nodos siguientes o previos.

Pero, en el caso de la matriz dispersa, la función cambia un poquito, ya que primero se abre el archivo .dot, y al recorrer la matriz, se va escribiendo en unas estructuras muy parecidas a pilas (de manera gráfica), y después ya se genera el archivo png de la misma.

```
#-----SUB PARA GRAFICAR LA MATRIZ DISPERSA-----
sub graficarMatrizMedicamentos {
    #lo haremos de una forma diferente a los otros grafos
    #CLAVE IMPORTANTE
    #LAS FILAS SON LOS LABORATORIOS
    #LAS COLUMNAS SON LOS MEDICAMENTOS
    my ($matriz) = @_;
    open my $fh, ">", "graficos/matrizDispersa.dot" or die $!;

    print $fh "digraph Matriz {\n";
    print $fh "    node [shape=record, fontname=\"Helvetica\"];\n";

    # Obtener todas las columnas (laboratorios)
    my @laboratorios = sort keys %{$matriz->{filas}};

    # Cabecera, serían los laboratorios
    my $header = "{ Laboratorio | " . join(" | ", @laboratorios) . " }";
    print $fh "    header [label=\"$header\"];\n";

    # Filas dinámicas, recorrer las medicina
    foreach my $med (sort keys %{$matriz->{columnas}}) {
        #se toma un item de la fila medicamento
        my $actual = $matriz->{columnas}{$med};
```

Estructura de carpetas

Esta sería una imagen simple de cómo está estructurado este proyecto.

